conventions. Developers are thus rewarded for doing the right thing, so it's less about giving up "your ways" and more about getting productivity for free.

Rails does this using a simple naming convention. If an incoming request has a controller named (say) `admin/book`, Rails will look for the controller called `book_controller` in the directory `app/controllers/admin`. That is, the final part of the controller name will always resolve to a file called *name*`_controller.rb`, and any leading path information will be used to navigate through subdirectories, starting in the `app/controllers` directory.

Imagine that our program has two such groups of controllers (say, `admin/`*xxx* and `content/`*xxx*) and that both groups define a book controller. There'd be a file called `book_controller.rb` in both the `admin` and `content` subdirectories of `app/controllers`. Both of these controller files would define a class named `BookController`. If Rails took no further steps, these two classes would clash.

To deal with this, Rails assumes that controllers in subdirectories of the directory `app/controllers` are in Ruby modules named after the subdirectory. Thus, the book controller in the `admin` subdirectory would be declared like this:

```
class Admin::BookController < ActionController::Base
  # ...
end
```

The book controller in the `content` subdirectory would be in the `Content` module:

```
class Content::BookController < ActionController::Base
  # ...
end
```

The two controllers are therefore kept separate inside your application.

The templates for these controllers appear in subdirectories of `app/views`. Thus, the following is the view template corresponding to this request:

```
http://my.app/admin/book/edit/1234
```

And it will be in this file: